

Akış Kontrol Mekanizmaları

- ***Koşul ifadeleri***
 - *if deyimi*
 - *switch deyimi*
- ***Döngü yapıları (loops)***
 - *for döngüsü*
 - *while döngüsü*
 - *foreach döngüsü*
- ***Atlama(jump)***
 - *break, continue, goto, return*

Koşul İfadeleri

- Programlar içersinde bazı durumlarda programın akışının değişmesi ya da farklı işlemlerin yapılması gerekebilir. Bazen de seçimlerin yapılması ya da belirli işlemlerin belli sayıda tekrar edilmesi gerekebilir.
- Bu gibi işlemleri gerçekleştirmek için koşul ifadeleri ve döngü yapıları kullanılır.
- C# dilinde koşula bağlı olarak işlemler gerçekleştirmek için iki farklı deyim kullanılabilir:
 - **if deyimi**
 - **switch deyimi**

Koşul İfadeleri

- Programlar içersinde bazı durumlarda programın akışının değişmesi ya da farklı işlemlerin yapılması gerekebilir. Bazen de seçimlerin yapılması ya da belirli işlemlerin belli sayıda tekrar edilmesi gerekebilir.
- Bu gibi işlemleri gerçekleştirmek için koşul ifadeleri ve döngü yapıları kullanılır.
- C# dilinde koşula bağlı olarak işlemler gerçekleştirmek için iki farklı deyim kullanılabilir:
 - **if deyimi**
 - **switch deyimi**

If Deyimi

- Program akış kontrol deyimlerinin başında gelir. Koşula göre değişik işlemlerin yapılmasını sağlar. Basit kullanımı:
 - *if(koşul)*
 - *Deyim1;*
 - *else*
 - *Deyim2;*
 - ya da
 - *if(koşul)*
 - *Deyim1;*
 - *else if*
 - *Deyim2;*
 - *else*
 - *Deyim3;*
- Eğer deyim kısmında 1 den fazla satır varsa { } program bloğu kullanmak zorunludur.

www.bilisimogretmeni.com

If Deyimi

```
• using System;
• class if_yapısı
• {
•     public static void Main()
•     {
•         string s;
•         int a,b,c;
•         Console.Write("1.sayıyı giriniz : ");
•         s=Console.ReadLine();
•         a=Int32.Parse(s);
•         Console.Write("2.sayıyı giriniz : ");
•         s=Console.ReadLine();
•         b=Int32.Parse(s);
•         Console.Write("3.sayıyı giriniz : ");
•         s=Console.ReadLine();
•         c=Int32.Parse(s);
•         if(a>=b&&a>=c) Console.WriteLine("Enbüyük :"+a);
•         else if(b>=c) Console.WriteLine("Enbüyük :"+b);
•         else Console.WriteLine("Enbüyük :"+c);
•     }
• }
```

If Deyimi

```
using System;
class if_3
{
    static void Main()
    {
        string secim;
        int s1, s2;

        Console.Write("1.Sayıyı Girin:");
        s1 = Convert.ToInt32(Console.ReadLine());
        Console.Write("2.Sayıyı Girin:");
        s2 = Convert.ToInt32(Console.ReadLine());

        Console.WriteLine("\nişleminizi Seçin:\n=====");
        Console.WriteLine("Toplama    (t)");
        Console.WriteLine("Çıkarma   (ç)");
        Console.WriteLine("Çarpma    (x)");
        Console.WriteLine("Bölme     (b)");
        Console.WriteLine("Mod Alma  (m)");
        secim = Console.ReadLine();

        if (secim == "t")
            Console.WriteLine("Toplam = {0}", s1 + s2);
        else if (secim == "ç")
            Console.WriteLine("Fark = {0}", s1 - s2);
        else if (secim == "x")
            Console.WriteLine("Çarpım = {0}", s1 * s2);
        else if (secim == "b")
        {
            if (s2==0)
                Console.WriteLine("Bölen 0 olamaz...");
            else
                Console.WriteLine("Bölüm = {0}", (float)s1 / (float)s2);
        }
        else if (secim == "m")
        {
            if (s2 == 0)
                Console.WriteLine("Bölen 0 olamaz...");
            else
                Console.WriteLine("Mod = {0}", s1 % s2);
        }
        else
            Console.WriteLine("Yanlış işlem Seçtiniz...");
    }
}
```

Switch Deyimi

- Bir ifadenin alabileceği değişik değerlere göre işlemler gerçekleştirmek için kullanılan deyimdir. Karmaşık if ifadelerini daha sade bir şekilde ifade etmeyi de sağlayabilir. Kullanımı:

- **Switch Deyimi**
- *Switch(ifade)*
- {
- *case sabit1:*
- *deyim1;*
- *break;*
- *case sabit2:*
- *deyim2;*
- *break;*
- *default:*
- *deyim3;*
- *break;*
- }

Switch Deyimi

- ▶ Her case ifadesinin bir atlama (**break, goto, return, continue**) ifadesi ile sonlandırmalısınız.
- ▶ **Continue** deyimi switch ifadesi bir döngünün gövdesinde ise kullanılabilir.
- ▶ Bir case içinde hiçbir ifade verilmezse atlama deyimi kullanılması gerekmez
 - ▶ Case 2: // ifade yok
 - ▶ Case 3:
 - ▶ Case 4: ifade....;break,;
 - ▶ Case 5: ifade...; goto case 4;

Switch Deyimi

- **switch ifadesi kullanırken dikkat edilecek bazı kurallar vardır:**
 - case sözcüğünden sonra gelen ifadeler sabit olmak zorundadır.
 - **case ifadeleri tamsayı, karakter ya da string sabitler olabilir.**
 - default ve case ifadeleri istenilen sırada yazılabilir.
 - Aynı switch bloğu içersinden birden fazla aynı case ifadesi bulunamaz.
 - default kullanmak zorunlu değildir.
 - Akışı bir case ifadesinden bir başka case ifadesine yönlendirmek için mutlaka goto anahtar sözcüğü kullanılır.

Switch Deyimi

```
• using System;
• class switch_case_yapısı{
•     public static void Main()      {
•         string s;
•         int a,b,c;
•         Console.Write("Kaçınıcı Sınıftasınız : ");
•         s=Console.ReadLine();
•         a=Int32.Parse(s);
•         switch(a)                  {
•             case(1): Console.WriteLine("Daha Yenisiniz");
•                     break;
•             case(2): Console.WriteLine("Mirasçısınız");
•                     break;
•             case(3): Console.WriteLine("Ev Sahibisiniz");
•                     break;
•             case(4): Console.WriteLine("Misafirsiniz");
•                     break;
•             default: Console.WriteLine("Siz okulu uzatmışsınız");
•                     break;
•         }
•     }
• }
```

Switch Deyimi

```
using System;

class switch_1
{
    static void Main()
    {
        Console.Write("Bir sayı girin:");
        int sayi = Convert.ToInt32(Console.ReadLine());

        switch (sayi)
        {
            case 1:
            case 2:
                Console.WriteLine("Case 1 ve 2");
                break;
            case 3:
                Console.WriteLine("Case 3");
            case 4:
                Console.WriteLine("Case 4");
                goto case 5;
            case 5:
                Console.WriteLine("Case 5");
                break;
            default:
                Console.WriteLine("Default");
                break;
        }
    }
}
```

Switch Deyimi

```
using System;

class SwitchSelect
{
    public static void Main()
    {
        string myInput;
        int myInt;

        begin:

        Console.Write("Please enter a number between 1 and 3: ");
        myInput = Console.ReadLine();
        myInt = Int32.Parse(myInput);

        // switch with integer type
        switch (myInt)
        {
            case 1:
                Console.WriteLine("Your number is {0}.", myInt);
                break;
            case 2:
                Console.WriteLine("Your number is {0}.", myInt);
                break;
            case 3:
                Console.WriteLine("Your number is {0}.", myInt);
```

Switch Deyimi

```
        break;
    default:
        Console.WriteLine("Your number {0} is not between 1 and 3.",
myInt);
        break;
    }

    decide:

    Console.Write("Type \"continue\" to go on or \"quit\" to stop: ");
    myInput = Console.ReadLine();

    // switch with string type
    switch (myInput)
    {
        case "continue":
            goto begin;
        case "quit": Console.WriteLine("Bye."); break;

        default:
            Console.WriteLine("Your input {0} is incorrect.", myInput);
            goto decide;
    }

}

}
```

Switch Deyimi

```
switch (myInt)
{
    case 1:
    case 2:
    case 3:
        Console.WriteLine("Your number is {0}.", myInt);
        break;
    default:
        Console.WriteLine("Your number {0} is not between 1 and 3.", myInt);
        break;
}
```

Döngüler(Loops)

- Program içerisinde belirli işleri tekrar tekrar yapılmasını sağlayan kod bloklarıdır. Döngüler sonsuz olabileceği gibi belli bir koşul ile kontrol etmek de mümkündür. C# dilinde dört tip döngü yapısı bulunur:
- for
- while
- do while
- foreach

For Döngüsü

- *for(ifade1;ifade2;ifade3)*
- *{*
- *deyimler;*
- *}*
- for döngü tanımlamasında iki “;” ile ayrılmış üç ifade bulunur. Bunlardan bazıları boş olabilir fakat mutlaka “;” işaretleri kullanılmalıdır.
- İlk ifade bir defaya mahsus olmak üzere çalıştırılır. Genelde döngü değişkeninin tanımlanması ya da ilk değer ataması için kullanılır.
- İkinci ifade ise döngünün kontrol edildiği kısımdır Buradaki ifade “true” değer ürettiği sürece döngü devam eder.
- Son ifade ise genelde döngü değişkeninin değerinin değiştirildiği kısımdır.

For Döngüsü

- *for(ifade1;ifade2;ifade3)*
- *{*
- *deyimler;*
- *}*
- Kullanımı:
- *for(;;)* Sonsuz döngüdür.
- *for(int i=1;i<=100;i++);*
- *for(int i=1,j=10;(i<=10)&(j>=1);i++,j--);*

For Döngüsü

```
using System;

class for_1
{
    static void Main()
    {
        Console.Write("Bir sayı girin:");
        int n = Convert.ToInt32(Console.ReadLine());
        int toplam = 0;

        for (int i = 1; i <= n; i++)
            toplam = toplam + i;

        Console.WriteLine("1\'den {0}\'e kadar olan sayıların toplamı {1}\'dir.", n, toplam);
    }
}
```

For Döngüsü

```
using System;

class for_2
{
    static void Main()
    {
        string s;

        for (s = Console.ReadLine(); s != "Çıkış";
            s = Console.ReadLine())

            Console.WriteLine(s);
    }
}
```

For Döngüsü

```
using System;

class for_3
{
    static void Main()
    {
        int i=0, a, n;

        Console.Write("Bir Sayı Girin :");
        n = Convert.ToInt32(Console.ReadLine());

        Console.Write("Artım Miktarı  :");
        a = Convert.ToInt32(Console.ReadLine());

        for (; i < n; )
        {
            Console.Write("{0} ", i);
            i += a;
        }
    }
}
```

For Döngüsü

```
using System;

class for_4
{
    static void Main()
    {
        int k, t, toplam, n1, n2;

        Console.Write("Aralık başlangıcı :");
        n1 = Convert.ToInt32(Console.ReadLine());

        Console.Write("Aralık Sonu      :");
        n2 = Convert.ToInt32(Console.ReadLine());

        if (n1 <= n2)
        {
            for (k = n1; k <= n2; k++)
            {
                toplam = 0;
                for (t = 1; t <= k; t++) { if (k % t == 0) toplam = toplam + t; }
                if (toplam == k + 1) Console.WriteLine(k);
            }
        }
        else
        {
            Console.WriteLine("Geçerli bir aralık giriniz...");
        }
    }
}
```

For Döngüsü

```
using System;

class for_5
{
    static void Main()
    {
        int sayi;

        Console.Write("Bir tamsayı giriniz :");
        sayi = Convert.ToInt32(Console.ReadLine());

        for (int bit = 32; bit >= 1; bit--)
        {
            Console.Write("{0}", (sayi >> bit - 1) & 1);
        }
        Console.WriteLine();
    }
}
```

While ve do-while Döngüsü

- *while (koşul)*
- {
- *deyimler*
- }
- Belirtilen koşul doğru olduğu sürece çalışmasına devam eder.

- ▶ *do while*
- ▶ {
- ▶ *deyimler*
- ▶ } *while(koşul);*
- ▶ for ve while döngülerinde koşul sağlanmadığı durumlarda döngü bloğu hiç çalıştırılmaz. Bazı durumlarda ise döngünün en az bir kez çalıştırılması gerekebilir. Bu durumda do while kullanılabilir.

While ve do-while Döngüsü

```
using System;

class while_1
{
    static void Main()
    {
        int i=0;

        while (i < 1000)
        {
            i += 5;
            Console.Write("{0,5}",i);
            if (i % 50 == 0) Console.WriteLine();
        }
    }
}
```


While ve do-while Döngüsü

```
using System;

class do_while_1
{
    static void Main()
    {
        int secim;

        do
        {
            Console.WriteLine("İşlemler\n=====");
            Console.WriteLine("1 - Toplama");
            Console.WriteLine("2 - Çıkarma");
            Console.WriteLine("3 - Çarpma");
            Console.WriteLine("4 - Bölme");
            Console.WriteLine("0 - Çıkış\n");
            Console.Write("İşleminizi Seçin:");
            secim = Convert.ToInt32(Console.ReadLine());
            switch (secim)
            {
                case 1: Console.WriteLine("Toplama işlemi seçildi."); break;
                case 2: Console.WriteLine("Çıkarma işlemi seçildi."); break;
                case 3: Console.WriteLine("Çarpma işlemi seçildi."); break;
                case 4: Console.WriteLine("Bölme işlemi seçildi."); break;
                case 0: Console.WriteLine("Çıkış seçildi."); break;
                default: Console.WriteLine("Yanlış seçim."); break;
            }
        }
        while(secim != 0);
    }
}
```

Foreach Döngüsü

- Farklı bir döngü yapısıdır Koleksiyon tabanlı nesneler içersinde adım adım dolaşılmasını sağlar.
- Bu döngü elemanlara tek tek ulaşım sağlar fakat ulaşılan elemanlar sadece okunabilir özelliğe sahiptir.

Foreach Döngüsü

- `string[ ] isimler = { "Ozlem", "Nesrin", "Ozge", "Fulya" };`
`//string dizi tanımı`

```
foreach (string ss in isimler) // elemanlar yazdırılıyor
{
    Console.WriteLine(ss);
}
```

- `foreach (int i in dizi) // i sadece okunur(readonly)`
`{ Console.WriteLine(i);`
`/* i++;*/ //yasaklı kullanılamaz`
`}`

Jump (atlama) Deyimleri)

- **Break:** Çalışan bir döngüden “break” sözcüğü kullanılarak çıkılabilir. Program akışı döngüden sonraki satırlardan devam eder. break sözcüğü sadece döngü ve switch ifadelerinde kullanılabilir.
- **Continue:** Döngünün bir sonraki " tekrarına geçilmesini sağlar.
- **Goto:** Programın etiket ile belirlenmiş herhangi bir kısmına atlamak için kullanılır. Nesneye yönelik programlamaya uygun bir yapı değildir. Switch ifadesindeki kullanımı dışında mümkün olduğunca kullanılmaktan kaçınılmalıdır.
- **Return :** Metotların herhangi bir anda sonlandırılması için kullanılır. Metot sonlandırıldıktan sonra programın akışı metodu çağıran fonksiyondan devam eder.

Jump (atlama) Deyimleri

```
using System;

class break_continue_1
{
    static void Main()
    {
        int sayi, t=0;

        while (true)
        {
            Console.Write("Negatif bir sayı girin:");
            sayi = Convert.ToInt32(Console.ReadLine());
            if (sayi == 0) break;
            if (sayi > 0)
            {
                Console.WriteLine("Pozitif sayı girdiniz,
tekrar deneyin...");
                continue;
            }
            t = t + sayi;
        }
        Console.WriteLine("Toplam :{0}", t);
    }
}
```

System.Random Sınıfı

- Rastgele sayı üretmek için .NET Framework sınıf kütüphanesindeki **System.Random** sınıfı kullanılır. Rastgele sayı üretmek için öncelikle **Random** sınıfı türünden bir nesne oluşturmak gerekir.
- *Random rnd=new Random ();//tanımlama*
- rnd nesnesi oluşturulduktan sonra artık bu nesnenin metotlarına erişebiliriz.

System.Random Sınıfı

- *Int* rastgelesayı = *rnd.next(10,20);*
 - 10 ile 20 arasında bir sayı üretir.
- *Int* rastgelesayı = *rnd.next(50);*
 - 0 ile 50 arasında bir sayı üretir.
- *Int* rastgelesayı = *rnd.next();*
 - Herhangi bir rasgele sayı üretir.
- *double* rastgelesayı = *rnd.NextDouble();*
 - 0.0 ile 1 arasında bir sayı üretir.

ÖRNEKLER

Sonuç:

$(a < b) \&\&(c < d) \rightarrow \text{False}$

$(a < b) \|\|(c < d) \rightarrow \text{True}$

$! (a < b) \rightarrow \text{False}$

$(a < b) \&(c < d) \rightarrow \text{False}$

$(a < b) \|(c < d) \rightarrow \text{True}$

$(a < b) \wedge (c < d) \rightarrow \text{True}$

- Örnek: Operatörler -Koşul
- using System;
- class Program {
- public static void Main(string [] args) {
- int a = 2 ; int b = 3 ; int c = 6 ; int d = 1 ;
- /* (a < b) = bu ifadenin doğru (true) olduğunu biliyoruz
- (c < d) = bu ifadenin yanlış (false) olduğunu biliyoruz */
-
- Console.WriteLine(" (a<b)&&(c<d) --> " + ((a<b)&&(c<d)));
- Console.WriteLine(" (a<b) \|\|(c<d) --> " + ((a<b) \|\|(c<d)));
- Console.WriteLine(" ! (a<b) --> " + (! (a<b)));
- Console.WriteLine(" (a<b)&(c<d) --> " + ((a<b)&(c<d)));
- Console.WriteLine(" (a<b) \|(c<d) --> " + ((a<b) \|(c<d)));
- Console.WriteLine(" (a<b)^(c<d) --> " + ((a < b) ^ (c < d)));
- }
- }

www.bilisimogretmeni.com

ÖRNEKLER

- Örnek: Operatörler -Bit
- using System;
- class Program {
- public static void Main(string [] args) {
- int a = 2 ; int b = 3 ; int c = 6 ;
-
- Console.WriteLine(" (a & b) --> " + (a & b));
- Console.WriteLine(" (a | b) --> " + (a | b));
- Console.WriteLine(" (a ^ b) --> " + (a ^ b));
- Console.WriteLine(" (~a) --> " + (~a));
- Console.WriteLine(" (~b) --> " + (~b));
- Console.WriteLine(" (~c) --> " + (~c));
- }
- }

Sonuç:

(a & b) --> 2
(a | b) --> 3
(a ^ b) --> 1
(~a) --> -3
(~b) --> -4
(~c) --> -7

ÖRNEKLER

- Örnek While yapısı
- using System;
- class Program {
- public static void Main(string [] args) {
- int i = 0 ; //döngü kontrol değişkeni
- while (i < 5) {
- Console.WriteLine("i = " + i);
- i++ ;
- }
- }
- }
- i = 0
- i = 1
- i = 2
- i = 3
- i = 4

ÖRNEKLER

- Örnek do-While
- using System;
- class Program {
- public static void Main(string [] args) {
-
- int i = 0 ; //döngü kontrol değişkeni
-
- do {
- Console.WriteLine("i = " + i);
- i++ ;
- } while (i < 0);
- }
- }
- i = 0

ÖRNEKLER

- Örnek: for
- using System;
- class Program {
- public static void Main(string [] args) {
-
- for(int i =0, j=0;i<20 ;i++,j++)
- { i*=j; Console.WriteLine("i = "+i+" j = "+j); }
-
- }
- }